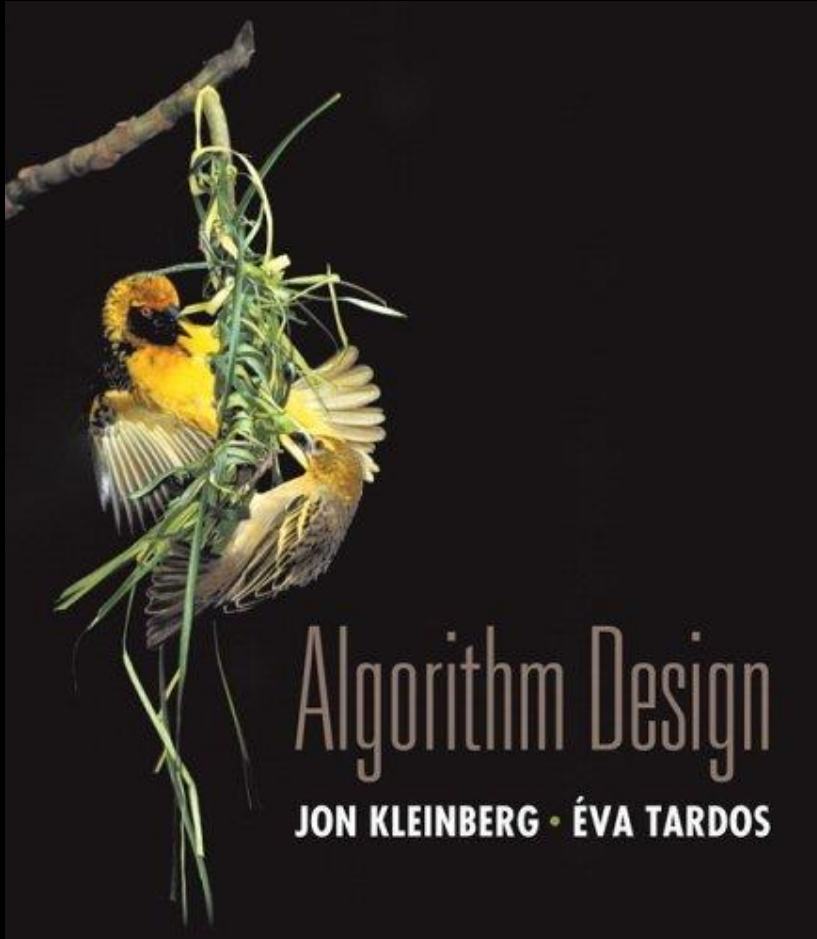


# Chapter 8

## NP e Computational Intractability



# Complessità Algoritmi

- **Indecidibilità** Nessun algoritmo possibile
- **Classe P**  $O(n^k)$  possibile
- **NP-completezza**  $O(n^k)$  non probabile

# Classificazione dei problemi in accordo alle richieste computazionali

**Domanda.** Quali problemi possiamo risolvere in pratica?

**Una definizione operativa.** Quelli che ammettono algoritmi aventi tempo polinomiale

# Classificazione dei problemi

**Desiderata.** Classificazione dei problemi in accordo un quelli che possono essere risolti in tempo polinomiale e quelli che non possono essere risolti in tempo polinomiale .

Abbiamo definito la classe P

**Non è stato possibile classificare per decenni tantissimi problemi fondamentali**

**Vedremo** che questi problemi sono "computazionalmente equivalenti"

# Riduzioni Polinomiali

**Desiderata.** Supponiamo che possiamo risolvere  $X$  in tempo polinomiale.  
Quali altri problemi possiamo risolvere in tempo polinomiale?

## Riduzioni Polinomiali

**Desiderata.** Supponiamo che possiamo risolvere  $X$  in tempo polinomiale.  
Quali altri problemi possiamo risolvere in tempo polinomiale?

# Riduzioni Polinomiali

**Desiderata.** Supponiamo che possiamo risolvere  $X$  in tempo polinomiale. Quali altri problemi possiamo risolvere in tempo polinomiale?

Non confondere con si riduce da

**Riduzione.** Problema  $X$  **si riduce in tempo polinomiale al** problema  $Y$  se arbitrarie istanze di  $X$  possono essere risolte usando:

- Un numero polinomiale di passi di computazione, più
- un numero polinomiale di chiamate ad un oracolo che risolve  $Y$ .

**Notazione.**  $X \leq_p Y$ .

Una "black box" che risolve istanze di  $Y$  in un solo passo

# Riduzioni Polinomiali

**Desiderata.** Supponiamo che possiamo risolvere  $X$  in tempo polinomiale. Quali altri problemi possiamo risolvere in tempo polinomiale?

Non confondere con si riduce da

**Riduzione.** Problema  $X$  **si riduce in tempo polinomiale al** problema  $Y$  se arbitrarie istanze di  $X$  possono essere risolte usando:

- Un numero polinomiale di passi di computazione, più
- un numero polinomiale di chiamate ad un oracolo che risolve  $Y$ .

**Notazione.**  $X \leq_p Y$ .

Una "black box" che risolve istanze di  $Y$  in un solo passo

**Nota.**

- Si usa tempo anche per scrivere l'istanza data in input alla black box  
 $\Rightarrow$  istanze di  $Y$  devono avere dimensione polinomiale.



# Riduzioni Polinomiali

**Scopo.** Classificare i problemi in accordo alla difficoltà **relativa**.

# Riduzioni Polinomiali

**Scopo.** Classificare i problemi in accordo alla difficoltà **relativa**.

**Fornire algoritmi.** Se  $X \leq_p Y$  e  $Y$  ammette soluzione in tempo polinomiale, allora anche  $X$  ammette soluzione in tempo polinomiale.

# Riduzioni Polinomiali

**Scopo.** Classificare i problemi in accordo alla difficoltà **relativa**.

**Fornire algoritmi.** Se  $X \leq_p Y$  e  $Y$  ammette soluzione in tempo polinomiale, allora anche  $X$  ammette soluzione in tempo polinomiale.

**Stabilire intrattabilità.** Se  $X \leq_p Y$  e  $X$  non ammette soluzione in tempo polinomiale, allora  $Y$  non ammette soluzione in tempo polinomiale.

# Riduzioni Polinomiali

**Scopo.** Classificare i problemi in accordo alla difficoltà **relativa**.

**Fornire algoritmi.** Se  $X \leq_p Y$  e  $Y$  ammette soluzione in tempo polinomiale, allora anche  $X$  ammette soluzione in tempo polinomiale.

**Stabilire intrattabilità.** Se  $X \leq_p Y$  e  $X$  non ammette soluzione in tempo polinomiale, allora  $Y$  non ammette soluzione in tempo polinomiale.

**Stabilire equivalenza.** Se  $X \leq_p Y$  e  $Y \leq_p X$ , allora usiamo  $X \equiv_p Y$ .

  
a meno del costo della riduzione



# Riduzione mediante equivalenza semplice

---

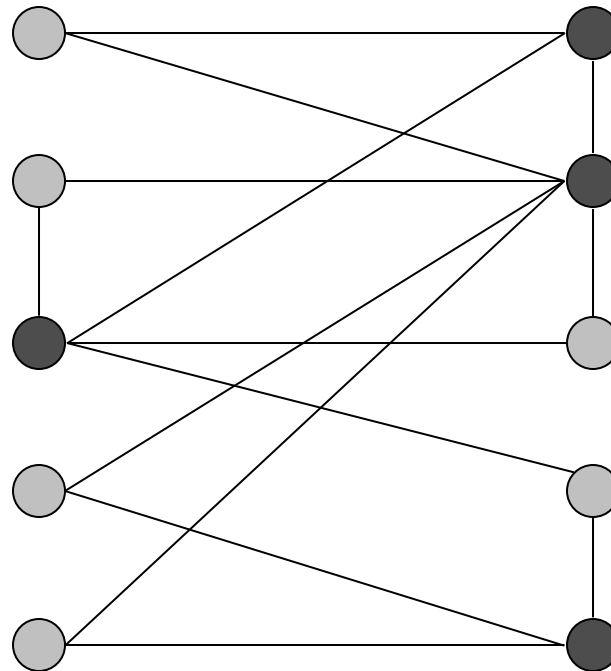
Riduzioni: strategie di base.

- Riduzione mediante equivalenza semplice.
- Riduzione da caso speciale a caso generale.
- Riduzione mediante codifica con gadgets.

# Independent Set (Insieme Indipendente)

**INDEPENDENT SET:** dato un grafo  $G = (V, E)$  e un intero  $k$ , esiste un sottinsieme di vertici  $S \subseteq V$  tale che  $|S| \geq k$ , e per ogni edge al più uno di suoi estremi è in  $S$ ?

**Es.** esiste un insieme independentedi dimensione  $\geq 6$ ?



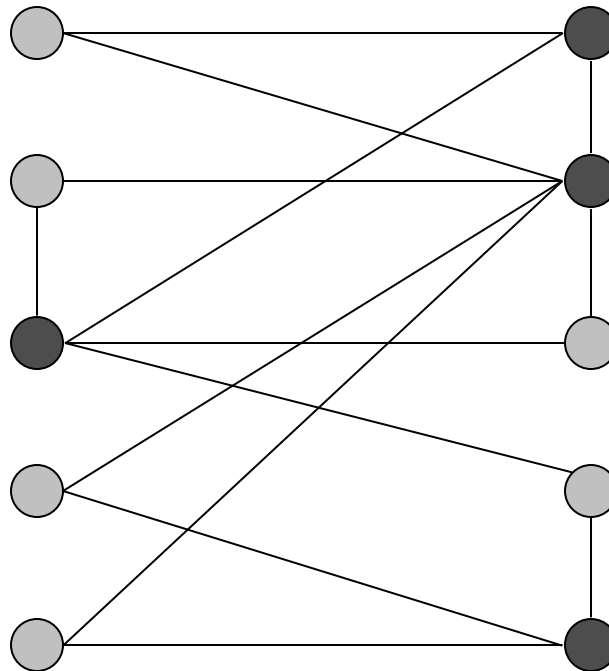
● independent set

# Independent Set (Insieme Indipendente)

**INDEPENDENT SET:** dato un grafo  $G = (V, E)$  e un intero  $k$ , esiste un sottinsieme di vertici  $S \subseteq V$  tale che  $|S| \geq k$ , e per ogni edge al più uno di suoi estremi è in  $S$ ?

**Es.** esiste un insieme indipendente di dimensione  $\geq 6$ ? **Si.**

**Es.** esiste un insieme indipendente di dimensione  $\geq 7$ ? **No.**



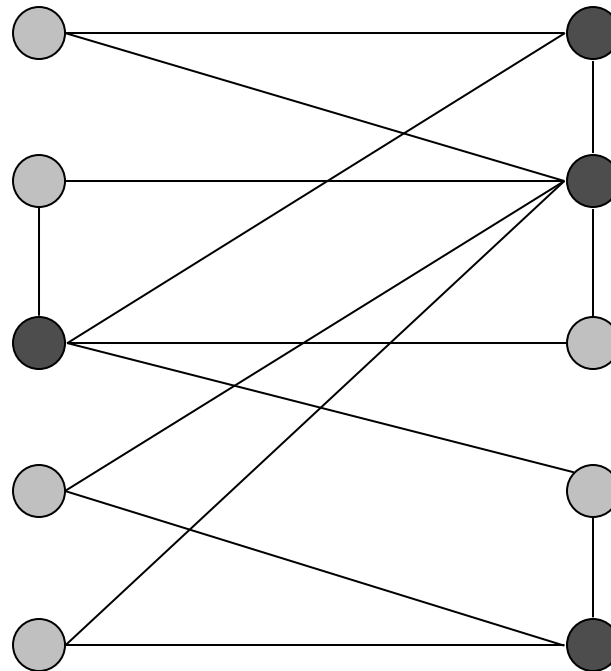
● independent set



# Vertex-cover

**Vertex-cover:** dato un grafo  $G = (V, E)$  e un intero  $k$ , esiste un sottinsieme di vertici  $S \subseteq V$  tale che  $|S| \leq k$ , e per ogni edge, al meno uno di suoi estremi è in  $S$ ?

**Ex.** esiste un vertex-cover di dimensione  $\leq 4$ ?



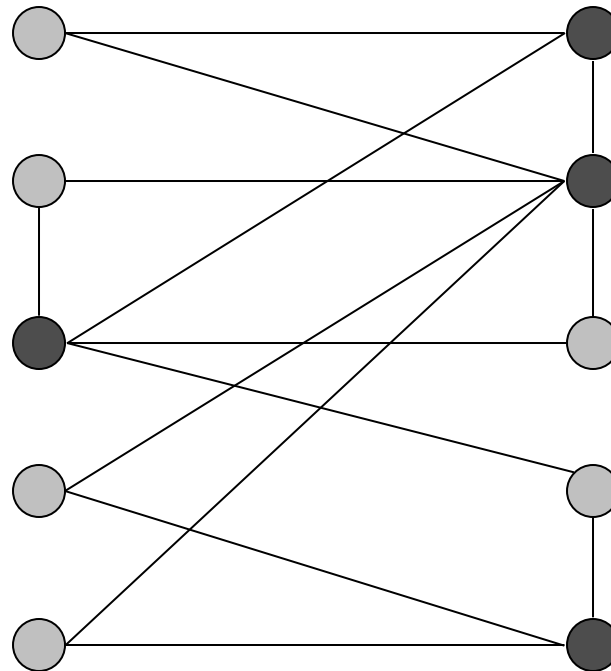
● vertex-cover

# Vertex-cover

**Vertex-cover:** dato un grafo  $G = (V, E)$  e un intero  $k$ , esiste un sottinsieme di vertici  $S \subseteq V$  tale che  $|S| \leq k$ , e per ogni edge, al meno uno di suoi estremi è in  $S$ ?

**Ex.** esiste un vertex-cover di dimensione  $\leq 4$ ? **Si.**

**Ex.** esiste un vertex-cover di dimensione  $\leq 3$ ? **No.**



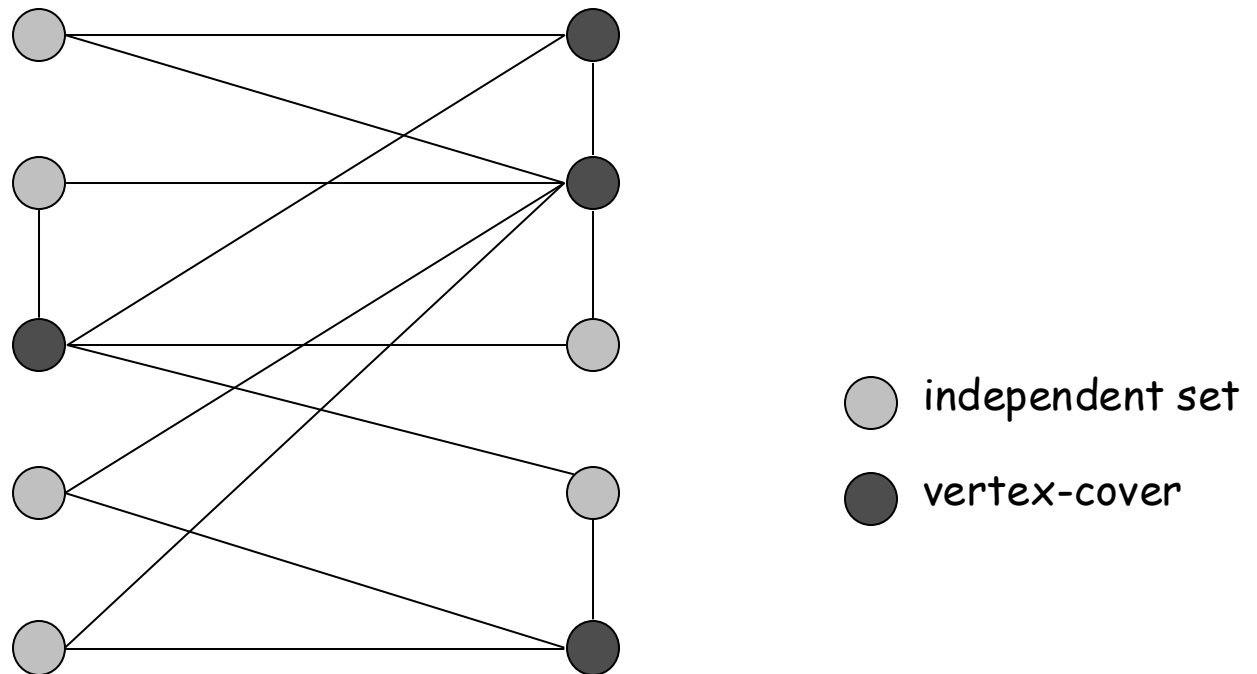
● vertex-cover

## Vertex-cover e Independent set

**Fatto.** VERTEX-COVER  $\equiv_p$  INDEPENDENT-SET.

**Dim.** Mostriamo che

$S$  è un insieme indipendente sse  $V - S$  è un vertex-cover.



## Vertex-cover e Independent set

**Fatto.** VERTEX-COVER  $\equiv_p$  INDEPENDENT-SET.

**Dim.** Mostriamo  $S$  è un insieme indipendente sse  $V - S$  è un vertex-cover.

## Vertex-cover e Independent set

**Fatto.** VERTEX-COVER  $\equiv_p$  INDEPENDENT-SET.

**Dim.** Mostriamo  $S$  è un insieme indipendente sse  $V - S$  è un vertex-cover.

$\Rightarrow$

- Sia  $S$  be un qualsiasi independent set.
- Consideriamo un arbitrario edge  $(u, v)$ .
- $S$  indipendente  $\Rightarrow u \notin S$  o  $v \notin S \Rightarrow u \in V - S$  o  $v \in V - S$ .
- Quindi,  $V - S$  copre  $(u, v)$ .

## Vertex-cover e Independent set

**Fatto.** VERTEX-COVER  $\equiv_p$  INDEPENDENT-SET.

**Dim.** Mostriamo  $S$  è un insieme indipendente sse  $V - S$  è un vertex-cover.

$\Rightarrow$

- Sia  $S$  be un qualsiasi independent set.
- Consideriamo un arbitrario edge  $(u, v)$ .
- $S$  indipendente  $\Rightarrow u \notin S$  o  $v \notin S \Rightarrow u \in V - S$  o  $v \in V - S$ .
- Quindi,  $V - S$  copre  $(u, v)$ .

$\Leftarrow$

- Sia  $V - S$  un qualsiasi vertex-cover.
- Consideriamo due nodi  $u \in S$  e  $v \in S$ .
- Notiamo che  $(u, v) \notin E$  poichè  $V - S$  è un vertex-cover.
- Quindi, non esistono due nodi in  $S$  connessi da un edge  
 $\Rightarrow S$  independent set.

# Riduzione da caso speciale a caso generale

---

## Strategie di riduzione

- Riduzione mediante equivalenza semplice.
- Riduzione da caso speciale a caso generale.
- Riduzione mediante codifica con gadgets.

# Set Cover

**SET-COVER:** dato un insieme  $U$  di elementi, una collezione  $S_1, S_2, \dots, S_m$  di sottoinsiemi di  $U$ , e un intero  $k$ , esiste una collezione di  $\leq k$  di questi insiemi la cui unione è uguale a  $U$ ?

$$U = \{ 1, 2, 3, 4, 5, 6, 7 \}$$

$$k = 2$$

$$S_1 = \{3, 7\}$$

$$S_4 = \{2, 4\}$$

$$S_2 = \{3, 4, 5, 6\}$$

$$S_5 = \{5\}$$

$$S_3 = \{1\}$$

$$S_6 = \{1, 2, 6, 7\}$$



# Set Cover

**SET-COVER:** dato un insieme  $U$  di elementi, una collezione  $S_1, S_2, \dots, S_m$  di sottoinsiemi di  $U$ , e un intero  $k$ , esiste una collezione di  $\leq k$  di questi insiemi la cui unione è uguale a  $U$ ?

$U = \{ 1, 2, 3, 4, 5, 6, 7 \}$

$k = 2$

$S_1 = \{3, 7\}$

$S_4 = \{2, 4\}$

$S_2 = \{3, 4, 5, 6\}$

$S_5 = \{5\}$

$S_3 = \{1\}$

$S_6 = \{1, 2, 6, 7\}$

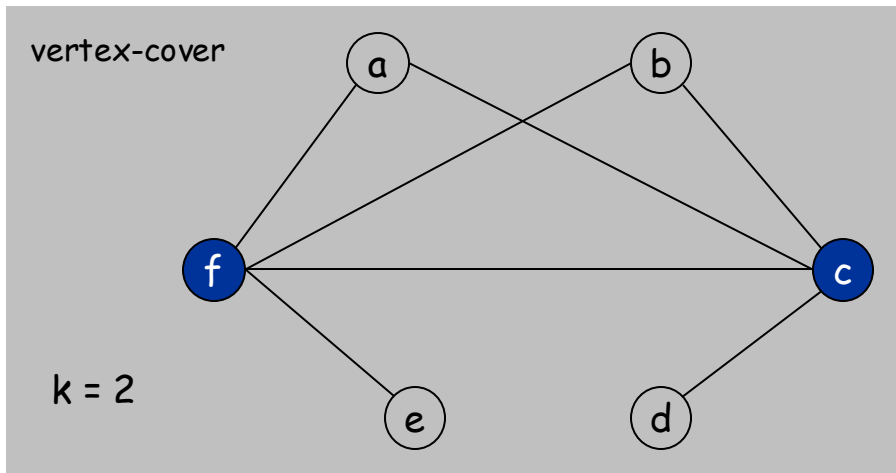
## Esempio di applicazione.

- $m$  parti di software disponibili
- Insieme  $U$  di  $n$  specifiche che desideriamo per il nostro sistema.
- La parte  $i$ -ma di software fornisce l'insieme  $S_i \subseteq U$  di specifiche.
- **Goal:** ottenere tutte le  $n$  specifiche usando il minimo numero di parti di software.

## Vertex-cover si riduce a Set-Cover

**Fatto.**  $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$ .

**Dim.** Data un'istanza di VERTEX-COVER  $G = (V, E)$ ,  $k$ ,  
costruiamo un'istanza di set-cover la cui dimensione è uguale alla dimensione  
dell'istanza di vertex-cover .



# Vertex-cover si riduce a Set-Cover

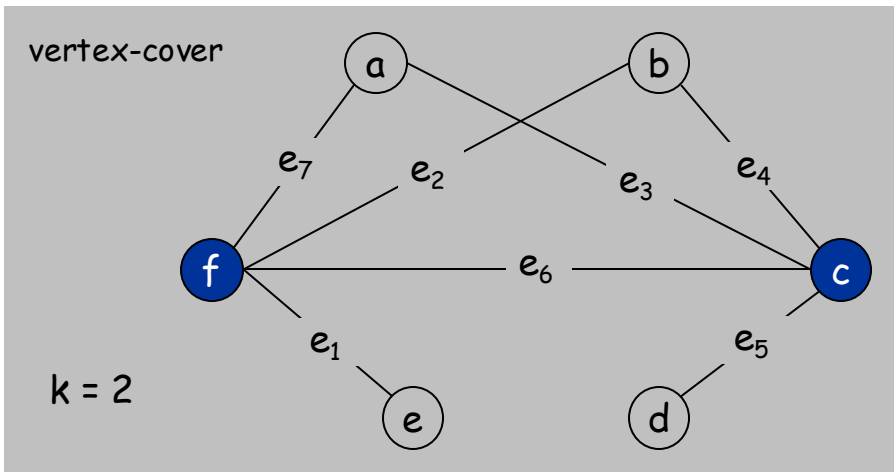
**Fatto.** VERTEX-COVER  $\leq_p$  SET-COVER.

**Dim.** Data un'istanza di VERTEX-COVER  $G = (V, E)$ ,  $k$ ,  
costruiamo un'istanza di set-cover la cui dimensione è uguale alla dimensione  
dell'istanza di vertex-cover.

## Costruzione.

- Creiamo istanza di SET-COVER :

$$k = k, \quad U = E, \quad S_v = \{e \in E : e \text{ incidente su } v\}$$



## SET-COVER

$$U = \{1, 2, 3, 4, 5, 6, 7\}$$

$$k = 2$$

$$S_a = \{3, 7\}$$

$$S_c = \{3, 4, 5, 6\}$$

$$S_e = \{1\}$$

$$S_b = \{2, 4\}$$

$$S_d = \{5\}$$

$$S_f = \{1, 2, 6, 7\}$$

## Vertex-cover si riduce a Set-Cover

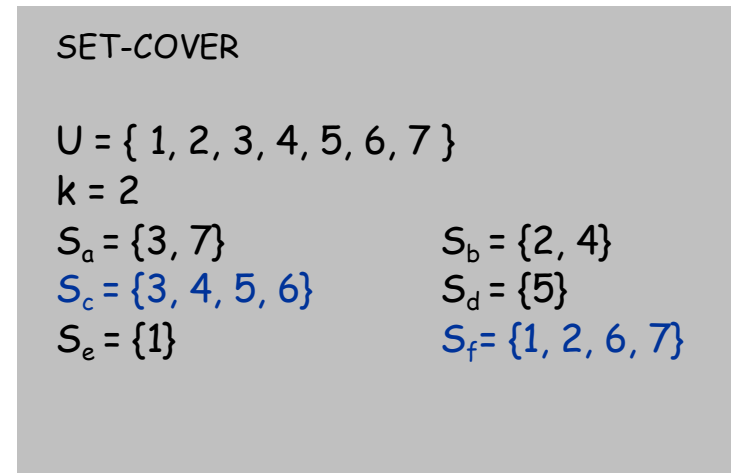
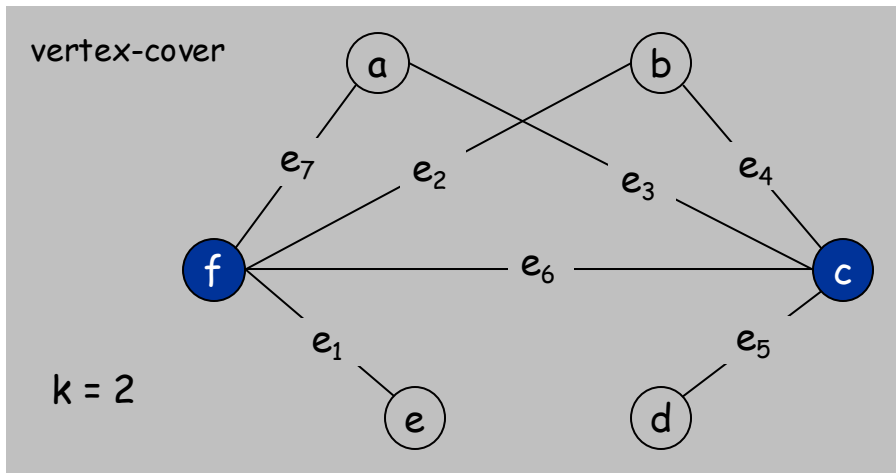
**Fatto.**  $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$ .

**Dim.** Data un'istanza di VERTEX-COVER  $G = (V, E)$ ,  $k$ ,  
costruiamo un'istanza di set-cover la cui dimensione è uguale alla dimensione  
dell'istanza di vertex-cover.

### Costruzione.

- Creiamo istanza di SET-COVER :

$$k = k, \quad U = E, \quad S_v = \{e \in E : e \text{ incidente su } v\}$$



Set-cover di dimensione  $\leq k$  sse vertex-cover di dimensione  $\leq k$ .

## 8.2 Riduzioni via "Gadgets"

---

### Strategie di riduzione

- Riduzione mediante equivalenza semplice.
- Riduzione da caso speciale a caso generale.
- Riduzione via "gadgets."

# Soddisfacibilità

**Letterale:** una variabile Booleana o la sua negazione.  $x_i$  or  $\overline{x_i}$

**Clausola:** un OR (o disgiunzione) di letterali.  $C_j = x_1 \vee \overline{x_2} \vee x_3$

**Forma Normale Congiuntiva (CNF):** una formula  $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$  proposizionale  $\Phi$  che è un AND (o congiunzione) di clausole.

**Es:** 
$$(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$$

Nota. Ogni formula logica può essere espressa in CNF

# Soddisfacibilità

**Letterale:** una variabile Booleana o la sua negazione.  $x_i$  or  $\overline{x_i}$

**Clausola:** un OR (o disgiunzione) di letterali.  $C_j = x_1 \vee \overline{x_2} \vee x_3$

**Forma Normale Congiuntiva (CNF):** una formula  $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$  proposizionale  $\Phi$  che è un AND (o congiunzione) di clausole.

**Es:**  $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$

**SAT:** data formula CNF  $\Phi$ , esiste un'assegnazione di verità che la soddisfa ?

↖  
Ognuno corrispondente ad una variabile diversa

**3-SAT:** SAT dove ogni clausola contiene esattamente 3 letterali.

**Si:**  $x_1 = \text{true}, x_2 = \text{true}, x_3 = \text{false}$ .

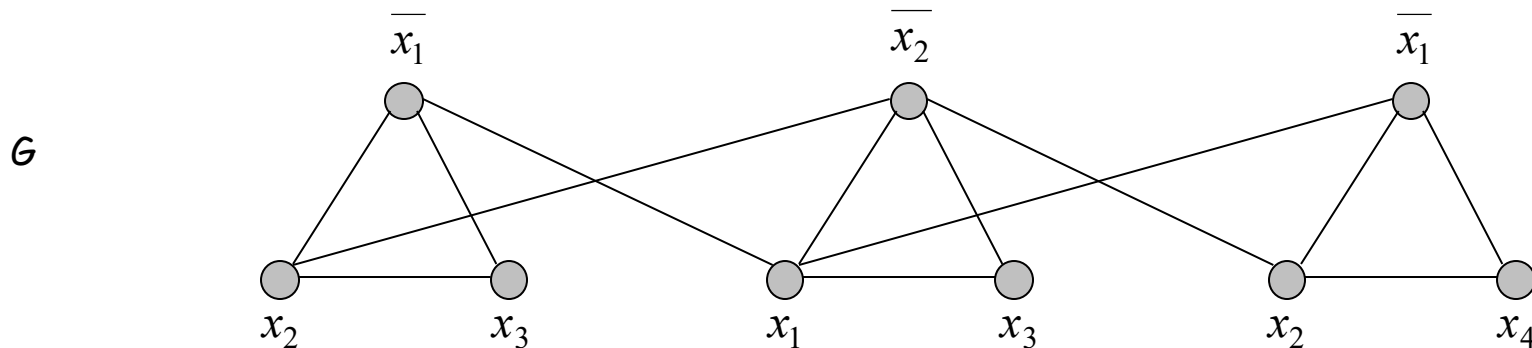
## 3-Soddisfacibilità si riduce a Independent set

**Fatto.**  $3\text{-SAT} \leq_p \text{INDEPENDENT-SET}$ .

**Dim.** data istanza  $\Phi$  di 3-SAT, costruiamo un' istanza  $(G, k)$  di INDEPENDENT-SET che ha un insieme indipendente di dimensione  $k$  sse  $\Phi$  è soddisfacibile.

**Costruzione.**

- $G$  contiene 3 vertici per ogni clausola, uno per ogni letterale.
- connettiamo i 3 letterali in un clausola in un triangolo.
- connettiamo ogni letterale a ogni suo negato.



$k = 3$

$$\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$$



### 3-SAT si riduce a Independent-Set

**Fatto.**  $G$  contiene insieme indipendente di dimensione  $k = |\Phi|$  sse  $\Phi$  è soddisfacibile.

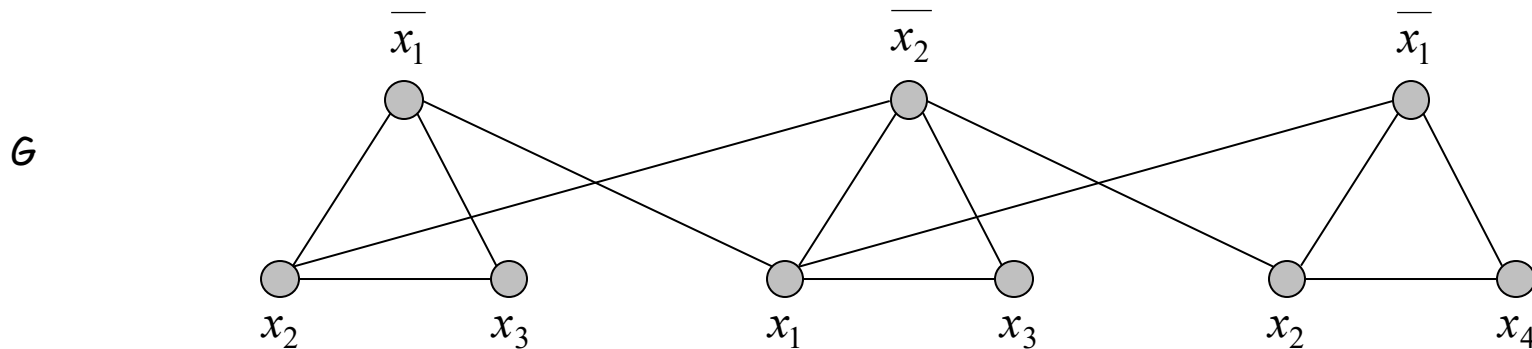
**Dim.**

$\Rightarrow$  Sia  $S$  l'insieme indipendente di dimensione  $k$ .

  $S$  deve contenere esattamente un vertice in ogni triangolo.

 Non può contenere un letterale ed il suo negato

- Poniamo questi letterali a true.
- Assegnazione di verità è consistente e tutte le clausole sono soddisfatte.



$k = 3$

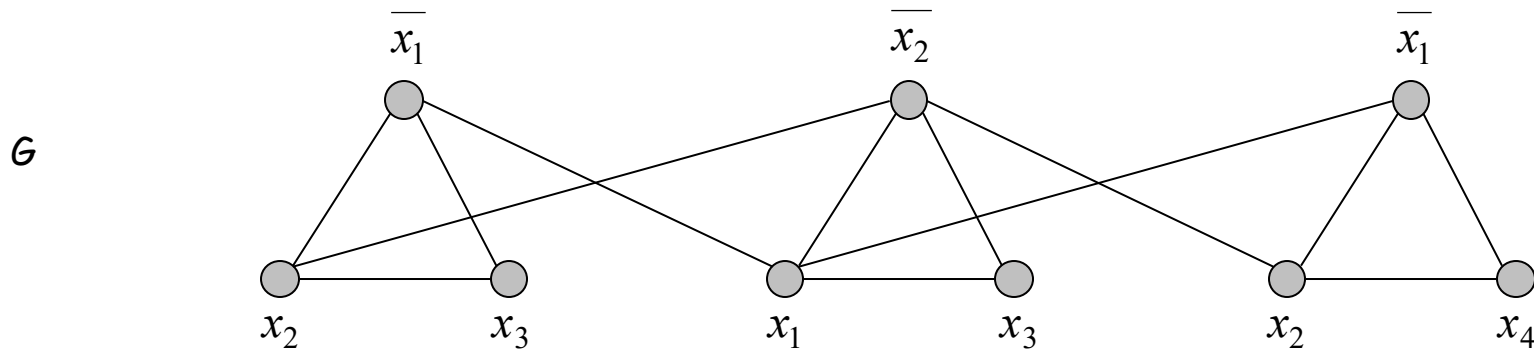
$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

### 3 Soddisfacibilità si riduce a Independent set

**Fatto.**  $G$  contiene insieme indipendente di dimensione  $k = |\Phi|$  sse  $\Phi$  è soddisfacibile.

**Dim.**

$\Leftarrow$  data un'assegnazione che soddisfa,  
selezioniamo un letterale true da ogni triangolo.  
Questo è un insieme indipendente di dimensione  $k$ . •



$k = 3$

$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

# Riepilogo

## Strategie viste.

- equivalenza semplice:  $\text{INDEPENDENT-SET} \equiv_p \text{VERTEX-COVER}$ .
- caso speciale a caso generale:  $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$ .
- Codifica con gadgets:  $3\text{-SAT} \leq_p \text{INDEPENDENT-SET}$ .

transitività. If  $X \leq_p Y$  e  $Y \leq_p Z$ , then  $X \leq_p Z$ .

Dim idea. Componiamo i due algoritmi.

Es:  $3\text{-SAT} \leq_p \text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER} \leq_p \text{SET-COVER}$ .